

Task-Driven Differentiable Active Depth Sensing with Split-Stem Policy Learning

Guodong Zhao, Zhijun Meng, and Jingjing Wang

Abstract—Learning-based drone policies increasingly benefit from differentiable dynamics and control, but the observations used for control are usually produced by a fixed sensing pipeline. Treating observation formation as fixed is limiting for active depth navigation, where projector power, exposure, and gain determine how glare, specular returns, weak reflectance, and motion blur affect task-relevant geometry. We present task-driven differentiable active depth sensing, a closed-loop framework that makes active-camera registers policy-controllable variables and maps ideal depth, camera settings, and local degradation regimes to observed depth and sensor-quality maps through a differentiable active-stereo response model. The resulting camera supervision is obtained through the image-formation model rather than from hand-designed exposure rules. To stabilize the coupled sensing-control problem, a split-stem recurrent policy is trained with differentiable teacher optimization, dataset aggregation (Dagger)-style online relabeling, and flight-only adaptation with a frozen camera branch. In a simulated drone aperture-crossing benchmark, the learned policy produces scene-conditioned camera behavior and improves success rate by 12.6% relative to the fixed-camera baseline.

Index Terms—active sensing, active depth camera, differentiable perception, drone navigation, dataset aggregation, split-stem policy learning

I. INTRODUCTION

Vision-based autonomous drones make control decisions from observations whose quality changes with motion, material, illumination, and sensor configuration. Recent learning-based flight policies and differentiable simulators have improved policy training by propagating analytical feedback through dynamics, trajectory objectives, and control updates [1]–[4]. However, the perception process that forms the policy input is often kept outside this differentiable loop. Camera images or depth maps are treated as exogenous observations, even though sensing conditions can determine whether task-relevant geometry is available to the controller. This separation is particularly limiting for active depth navigation: when depth near an aperture edge, reflective surface, or low-reflectance region is degraded, the controller can react to a poor observation but cannot directly improve the acquisition process that produced it. The mismatch between differentiable control and fixed observation formation raises a broader question for differentiable perception: how should sensing itself be modeled, optimized, and connected to closed-loop robot learning?

Related Work

From a differentiable-perception perspective, the key issue is where the optimization graph begins. Differentiable simulation and differentiable control make analytical feedback increasingly available to robot learning [1]–[5]. In parallel,

camera simulation and embodied visual simulators provide rich observation-generation pipelines for policy learning and evaluation [6]–[9]. Image systems simulation and sensor-transfer studies further show that camera parameters, camera effects, and sensor-specific artifacts can materially alter downstream perception and validation [10]–[14]. Camera-simulation and sensor-transfer results support the premise that sensing should not be treated as a transparent measurement device. The primary role of camera simulators and sensor-transfer methods, however, is usually data generation, robustness testing, sensor transfer, or fixed camera design. Online camera acquisition remains less developed: the camera state must change as the robot approaches a task-relevant decision.

Differentiable camera models and differentiable rendering bring observation formation closer to optimization. Neural Camera Simulators learn controllable camera transformations, and DiffPhysCam builds a differentiable physics-based camera model for inverse rendering and embodied artificial intelligence [15], [16]. Task-Specific Camera Optimization with Simulation (TaCOS) further studies task-specific camera optimization by co-optimizing camera settings with simulation for a vision task [17]. Neural Radiance Fields (NeRF), Instant Neural Graphics Primitives (Instant-NGP), Nerfstudio, three-dimensional (3D) Gaussian Splatting, Surface-Aligned Gaussian Splatting (SuGaR), and NVDiffRecMC provide powerful gradients through scene geometry, appearance, and material reconstruction [18]–[23]. Depth-of-field and lens-aware variants further make optical parameters differentiable inside the rendering process [24]–[28]. Differentiable camera and rendering methods are important foundations for differentiable perception, but their usual objectives are reconstruction quality, novel-view synthesis, inverse rendering, or offline camera design. For active depth control, rendering is not sufficient as the final objective. It must serve as a geometric substrate for an observation-formation process that determines the depth actually used by camera supervision and flight.

Active perception and perception-aware drone navigation explicitly recognize that sensing and action should be designed together [29]–[36]. Classical active vision asks where to look, while recent flight systems adapt trajectories, viewpoints, or speed so that the current sensor can support the next motion decision. High-speed flight and drone-racing systems further show that perception and action must be co-designed under short reaction time [37]–[40]. Trajectory- and speed-adaptive perception changes robot motion around the available sensing stream. A complementary question is whether the robot should also adapt the acquisition state of the sensor itself, for example projector power, exposure, and gain in an active depth camera. Register-adaptive sensing matters because some failures are

better addressed by changing how the sensor acquires depth than by changing only where or how fast the robot moves.

Learning with an adaptive sensor also changes the training problem. Imitation learning methods such as dataset aggregation (Dagger) reduce policy drift by querying supervision on the learner-induced state distribution [41]. With active sensing, this distribution shift is not only a shift over vehicle states. It is also a shift over sensor-produced observations, because camera actions change the depth stream that the flight policy later consumes. A shared representation can therefore mix flight-control features with degradation cues needed by the camera branch, while an offline camera policy can be queried under observations it did not label. The resulting representation-interference risk motivates separating camera semantics from final flight adaptation rather than training a single monolithic policy head.

Motivation and Contributions

This study therefore targets a missing link in differentiable drone autonomy: the acquisition process that turns camera settings and scene conditions into the depth stream used for control. We treat a D455-like active-stereo depth camera as part of the closed-loop learning system rather than as a fixed input generator. A geometric rendering stage first provides ideal metric depth. A differentiable active-stereo response stage then maps projector power, exposure, gain, and local degradation regimes to observed depth and sensor-quality maps. The response model is intentionally task-oriented: it is designed to provide useful gradients for camera supervision under navigation-relevant depth degradation, not to reproduce every pixel-level detail of a commercial sensor.

The second design requirement is to stabilize learning once the sensor becomes adaptive. The camera branch must learn sensing semantics from differentiable teacher labels, but the flight branch must later adapt to the observation distribution produced by the learned camera. We therefore use a split-stem recurrent policy and a staged training procedure. The flight branch extracts geometry for acceleration control, while the camera branch uses an independent visual stem, camera-state input, and local-motion descriptor to predict the next absolute camera target. A state-wise differentiable teacher first solves for camera settings that improve local depth reliability. DAgger-style online relabeling then mitigates the distribution shift induced by the learned camera [41]. Finally, the camera branch is frozen while the flight branch adapts to the new observation stream.

The paper makes four contributions.

- We formulate active depth acquisition as a task-driven differentiable perception problem within closed-loop drone learning, making projector power, exposure, and gain policy-controllable sensing actions rather than fixed pre-processing constants.
- We develop a lightweight active-stereo response model that maps ideal metric depth and camera settings to observed depth and sensor-quality maps under localized degradation regimes, providing gradient-based camera supervision without requiring a full sensor-specific renderer.

- We introduce a split-stem recurrent policy and a staged training protocol that combines differentiable teacher optimization, DAgger-style relabeling, and flight-only adaptation to preserve camera semantics while improving closed-loop flight.
- We validate the framework in a controlled aperture-crossing scenario that stresses task-relevant depth under glare, specular reflection, and low-reflectance degradation, with comparisons to fixed, random-fixed, non-differentiable learned-camera, and blind baselines.

II. METHOD OVERVIEW

This section specifies the closed-loop learning system summarized in Fig. 1. At each control step, the vehicle receives an observed depth image generated by the current camera registers, predicts both a flight command and the next camera target, and then advances the simulator with exposure-dependent timing. Let $\mathbf{x}_t$ be the quadrotor state, $\mathbf{c}_t = [P_t, E_t, G_t]^\top$ the active-depth camera state, and $\rho_t \in \{\text{glare}, \text{specular}, \text{dark}\}$ the local sensing regime used in the validation scenes. The proposed method learns a closed-loop map

$$\Pi_\theta : (D_t^{\text{obs}}, \mathbf{s}_t, \mathbf{c}_t, \mathbf{h}_t, \mathbf{q}_t) \mapsto (\mathbf{u}_t, \mathbf{c}_{t+1}^*, \mathbf{h}_{t+1}, \mathbf{q}_{t+1}), \quad (1)$$

where $\mathbf{u}_t$ is a local acceleration command and $\mathbf{c}_{t+1}^*$ is an absolute camera target. The main technical choice is to represent the sensor as a differentiable module $\mathcal{S}_\phi$ and to train the camera branch through teacher optimization and online relabeling, instead of relying on sparse collision gradients alone. The remainder of the section defines the closed-loop differentiable system, the active-depth image-formation model, the split-stem policy, the camera teacher and relabeling procedure, and the final flight-only adaptation objective.

A. Closed-Loop Differentiable System

Fig. 2 shows the dependencies that make camera parameters part of the differentiable rollout rather than an external pre-processing choice. We denote by $\mathbf{x}_t$ the full simulator state at control step t , including vehicle pose, velocity, attitude, and collision margin. The symbol $\mathcal{E}$ denotes the environment description, including scene geometry, material attributes, light sources, and the randomized task parameters. The aperture-crossing scenario used for validation is one instantiation of $\mathcal{E}$; the formulation itself only assumes an environment that can provide geometry and sensing-regime parameters. The active depth camera state is

$$\mathbf{c}_t = [P_t, E_t, G_t]^\top \in [0, 1]^3, \quad (2)$$

where P_t , E_t , and G_t are the normalized projector power, exposure, and gain. The variable ρ_t indexes the local sensing condition induced by $\mathcal{E}$, such as glare, specular reflection, or low-reflectance material.

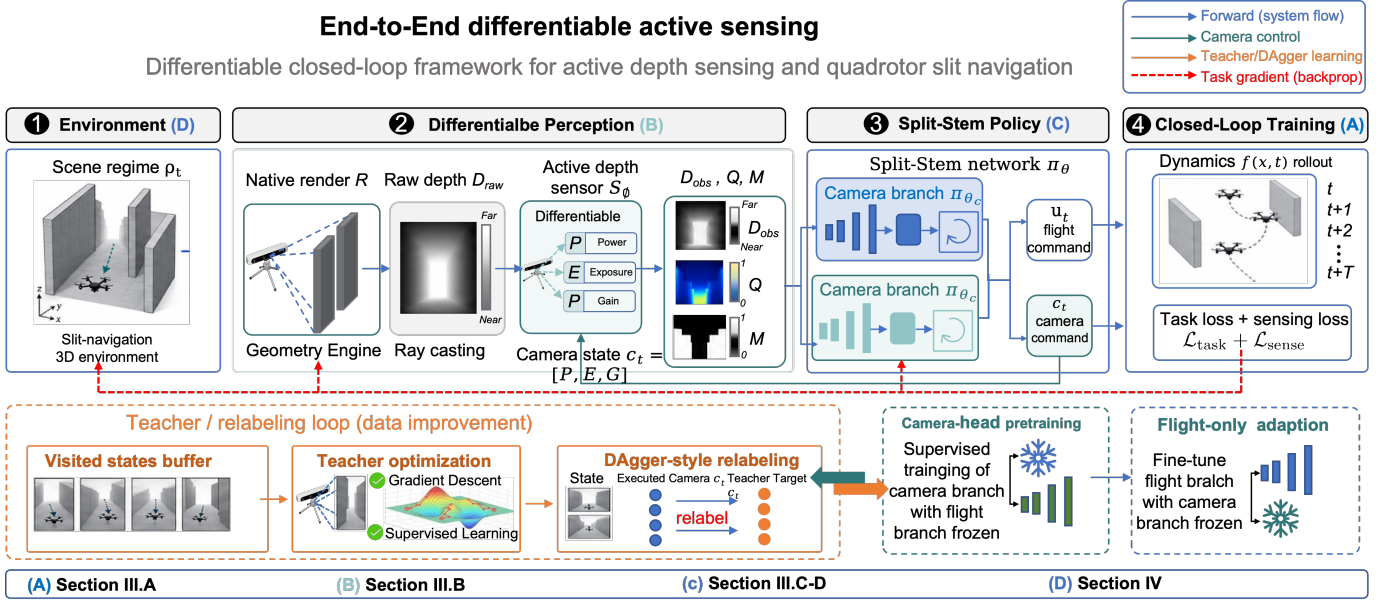


Fig. 1. Validation task and training protocol. A controlled aperture-crossing scenario instantiates the broader idea of task-driven differentiable perception with an active depth camera under localized degradation regimes. The policy controls both flight and camera parameters. Online states are relabeled by a differentiable camera teacher, the camera branch is pretrained on those labels, and final flight adaptation freezes the camera branch while updating the flight branch.

The closed-loop rollout is written as

$$D_t^{\text{raw}} = \mathcal{R}(\mathbf{x}_t, \mathcal{E}), \quad (3)$$

$$(D_t^{\text{obs}}, Q_t^{\text{obs}}) = \mathcal{S}_\phi(D_t^{\text{raw}}, \mathbf{c}_t, \rho_t, \mathbf{x}_t), \quad (4)$$

$$\mathbf{s}_t = \psi_{\text{flight}}(\mathbf{x}_t, \mathbf{x}_{\text{goal}}, \mathbf{c}_t), \quad (5)$$

$$\mathbf{m}_t = \psi_{\text{cam}}(\mathbf{x}_t, \mathbf{c}_t), \quad (6)$$

$$(\mathbf{u}_t, \mathbf{c}_{t+1}^*, \mathbf{h}_{t+1}, \mathbf{q}_{t+1}) = \Pi_\theta(D_t^{\text{obs}}, \mathbf{s}_t, \mathbf{c}_t, \mathbf{m}_t, \mathbf{h}_t, \mathbf{q}_t), \quad (7)$$

$$\mathbf{c}_{t+1} = \alpha \mathbf{c}_t + (1 - \alpha) \mathbf{c}_{t+1}^*, \quad \alpha = 0.7, \quad (8)$$

$$\mathbf{x}_{t+1} = f_{\text{dyn}}(\mathbf{x}_t, \mathbf{u}_t, \Delta t_t). \quad (9)$$

Here $\mathcal{R}$ is a geometric ray-intersection operator and $D_t^{\text{raw}} \in \mathbb{R}^{H \times W}$ is the ideal metric depth before sensor degradation. The differentiable path needed by the active camera is provided by the subsequent sensor-response model, which converts this ideal geometry into an observed depth image under explicit camera settings. The active-depth sensor $\mathcal{S}_\phi$ maps this ideal depth and the camera state to the policy observation D_t^{obs} and an observed quality map Q_t^{obs} . Reliability summaries used by the teacher and the loss are computed downstream from this sensor response, rather than exposed as separate policy inputs. The parameters ϕ collect the sensor-model constants, including range attenuation, exposure-time mapping, gain mapping, motion-blur and noise surrogates, and scene-effect masks.

The encoder ψ_{flight} builds the flight state $\mathbf{s}_t$ from goal-relative local velocity, body attitude, safety margin, and the current camera state when camera-state observation is enabled. In contrast, ψ_{cam} builds the camera-motion descriptor $\mathbf{m}_t$ from local velocity and attitude together with $\mathbf{c}_t$, but it does not use the goal vector. This design prevents the camera branch from learning a privileged schedule tied to the nominal validation geometry or goal location; camera commands must be conditioned on image quality, current registers, and motion. The policy Π_θ contains two recurrent states: $\mathbf{h}_t$ for flight

control and $\mathbf{q}_t$ for camera control. The camera target $\mathbf{c}_{t+1}^*$ is smoothed by the first-order update (8); we set $\alpha = 0.7$. The control interval satisfies

$$\Delta t_t = \Delta t_{\text{base}} + \kappa_\tau \tau(E_t), \quad (10)$$

where $\tau(E_t)$ is the effective exposure time and κ_τ is a small scaling factor. Thus, camera settings affect the closed loop both through the observation model and through the timing of the rollout.

For a horizon T , the differentiable training objective can be written as

$$\mathcal{J}(\theta) = \sum_{t=0}^{T-1} \ell_{\text{nav}}(\mathbf{x}_t, \mathbf{u}_t) + \ell_{\text{sens}}(r_t^{\text{fill}}, \mathbf{c}_t, \mathbf{x}_t). \quad (11)$$

The navigation loss ℓ_{nav} contains velocity tracking, smoothness, and collision terms, while ℓ_{sens} regularizes depth fill health and camera operation. Because D_t^{obs} is generated by $\mathcal{S}_\phi$ rather than treated as an exogenous input, a downstream task loss can provide a camera gradient. A one-step dependency is

$$\begin{aligned} \frac{\partial \mathcal{J}}{\partial \mathbf{c}_t} &= \frac{\partial \mathcal{J}}{\partial D_t^{\text{obs}}} \frac{\partial D_t^{\text{obs}}}{\partial \mathbf{c}_t} + \frac{\partial \mathcal{J}}{\partial r_t^{\text{fill}}} \frac{\partial r_t^{\text{fill}}}{\partial \mathbf{c}_t} \\ &+ \frac{\partial \mathcal{J}}{\partial \mathbf{x}_{t+1}} \frac{\partial \mathbf{x}_{t+1}}{\partial \mathbf{u}_t} \frac{\partial \mathbf{u}_t}{\partial D_t^{\text{obs}}} \frac{\partial D_t^{\text{obs}}}{\partial \mathbf{c}_t} + \frac{\partial \mathcal{J}}{\partial \Delta t_t} \frac{\partial \Delta t_t}{\partial \mathbf{c}_t}. \end{aligned} \quad (12)$$

The first two terms describe sensing feedback through observed depth and the downstream fill-health statistic r_t^{fill} . The third term describes task-level feedback that passes through the policy and dynamics. The final term captures the exposure-dependent timing effect. If the sensor response cannot provide derivatives with respect to $\mathbf{c}_t$, these gradients collapse to a fixed-sensor formulation. Equation (12) is therefore the mathematical motivation for differentiable active perception.

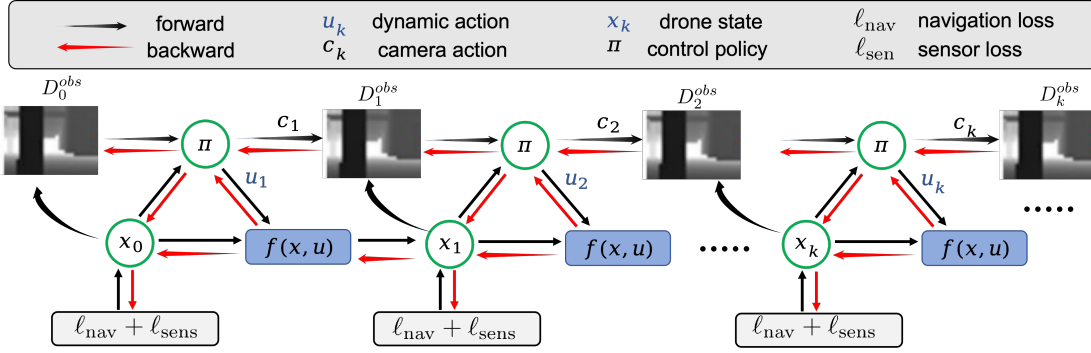


Fig. 2. Closed-loop differentiable system. The native geometric renderer produces ideal metric depth, the active-depth sensor response maps camera registers to observed depth, and the recurrent policy outputs both a flight command and the next camera target. The camera update and exposure-dependent rollout timing make sensing an active part of the control loop.

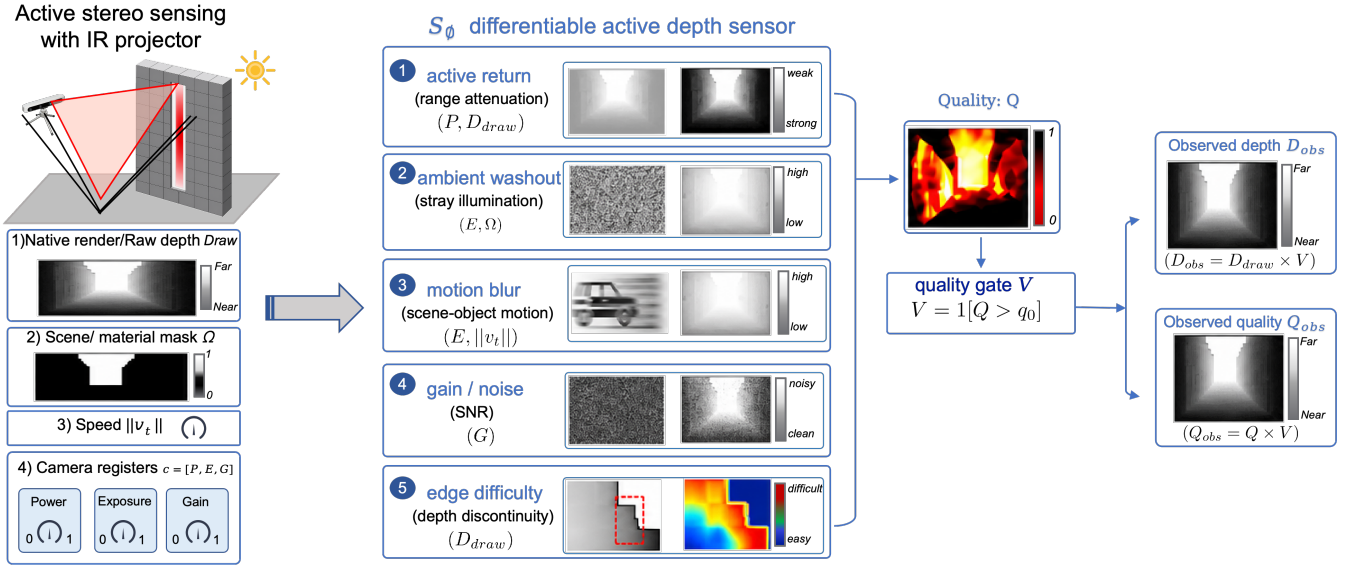


Fig. 3. Differentiable active-depth image formation. Ideal metric depth is combined with camera registers, motion, range attenuation, and localized sensing regimes to produce observed depth and a sensor-quality response. Downstream depth-health statistics and teacher losses are computed from this response rather than returned as additional policy inputs.

B. Differentiable Active-Depth Model

Fig. 3 summarizes the image-formation path from ideal metric depth to policy-visible degraded depth. The active-depth model follows the same modular principle as recent physics-based differentiable camera simulators: a geometric renderer first produces ideal scene measurements, and a differentiable camera model then converts them into sensor observations under explicit camera settings [16]. Our model is not a full photorealistic camera pipeline. It is a task-oriented active-stereo proxy that keeps the physical failure modes most relevant to agile local navigation through depth-critical bottlenecks: range-dependent active return, ambient infrared washout, gain-dependent noise, motion blur, depth-edge matching failure, and material-dependent degradation.

An active stereo depth camera such as a D455-like sensor emits an infrared pattern, observes the reflected pattern with stereo imagers, and estimates depth by local correspondence. The returned depth therefore depends not only on geometry, but also on projector power, exposure, gain, surface

reflectance, ambient infrared illumination, and vehicle motion. We write the raw geometric depth as $Z_t \equiv D_t^{\text{raw}}$ and decompose the model into a native geometric stage and a differentiable sensor stage.

Geometric depth. Let $R_t^{\mathcal{E}} \in SO(3)$ and $\mathbf{p}_t^{\mathcal{E}}$ be the camera orientation and position in the environment frame after applying the fixed body-to-camera transform and the scene-frame transform. Denote the three columns of $R_t^{\mathcal{E}}$ by $(\mathbf{r}_{x,t}, \mathbf{r}_{y,t}, \mathbf{r}_{z,t})$. For pixel (i, j) , where $i = 0, \dots, H - 1$ and $j = 0, \dots, W - 1$, the ray direction is

$$\eta_i = \left(2 \frac{i + 1/2}{H} - 1\right) f_y - \epsilon_r, \quad f_y = f_x \frac{H}{W}, \quad (13)$$

$$\xi_j = \left(2 \frac{j + 1/2}{W} - 1\right) f_x - \epsilon_r, \quad (14)$$

$$\mathbf{d}_{tij} = \mathbf{r}_{x,t} - \eta_i \mathbf{r}_{z,t} - \xi_j \mathbf{r}_{y,t}, \quad (15)$$

where FOV_x denotes the horizontal field of view. We set $f_x = \tan(\text{FOV}_x/2)$, and ϵ_r is a small numerical offset. The

ideal depth is the nearest positive ray intersection with the environment geometry.

$$Z_{tij} = \min_{\lambda > 0} \{ \lambda \mid \mathbf{p}_t^\varepsilon + \lambda \mathbf{d}_{tij} \in \mathcal{G}_\varepsilon \}. \quad (16)$$

The set $\mathcal{G}_\varepsilon$ contains the simulator primitives used by the validation scenes, including voxels, cylindrical obstacles, spherical obstacles, walls, and other drones when present. Equation (16) therefore defines the metric geometry that an ideal pinhole depth sensor would see before active infrared failures are applied.

Differentiable active-stereo response. The sensor stage first clamps the native depth into the valid measurement range,

$$\bar{Z}_{tij} = \text{clip}(Z_{tij}, z_{\min}, z_{\max}), \quad (17)$$

and computes a local depth-discontinuity score from the 3×3 pixel neighborhood $\mathcal{N}_{ij}$.

$$Z_{tij}^+ = \max_{(m,n) \in \mathcal{N}_{ij}} \bar{Z}_{tmn}, \quad (18)$$

$$Z_{tij}^- = \min_{(m,n) \in \mathcal{N}_{ij}} \bar{Z}_{tmn}, \quad (19)$$

$$\delta_{tij} = \text{clip} \left(\frac{Z_{tij}^+ - Z_{tij}^-}{\bar{Z}_{tij} + \epsilon_z}, 0, 1 \right). \quad (20)$$

The term δ_{tij} approximates the correspondence difficulty near occlusion boundaries and narrow-aperture edges. The environment also projects a continuous material or illumination mask

$$\Omega_{tij} = m_{\rho_t}(\bar{Z}_{tij}, \mathbf{x}_t, \mathcal{E}) \in [0, 1], \quad (21)$$

where $\Omega_{tij} = 0$ denotes ordinary material and larger values denote a localized sensing hazard. In the aperture-crossing validation scenes, side-wall masks are computed from the ray hit location in the scene frame, while glare masks are localized around the aperture and its halo. The pose used in this mask projection is treated as constant during sensor-gradient computation; this keeps the active-sensing gradient focused on the camera registers rather than on discontinuous ray-intersection topology.

The normalized camera command is $\mathbf{c}_t = [P_t, E_t, G_t]^\top$. We write $p_t = \text{clip}(P_t, 0, 1)$, $e_t = \text{clip}(E_t, 0, 1)$, and $g_t = \text{clip}(G_t, 0, 1)$. The exposure time and ISO-like gain are

$$\tau(E_t) = t_{\min} + t_{\text{span}} e_t, \quad (22)$$

$$\Gamma(G_t) = \Gamma_0 + \Gamma_s \frac{(g_t + \epsilon_g)^\gamma - \epsilon_g^\gamma}{(1 + \epsilon_g)^\gamma - \epsilon_g^\gamma}. \quad (23)$$

Here τ controls how long the infrared images accumulate photons and Γ controls the amplification of the received signal. The active and passive infrared returns are modeled as

$$S_{tij}^{\text{act}} = \frac{k_a p_t \tau(E_t)}{\bar{Z}_{tij}^2 + c_d}, \quad (24)$$

$$S_t^{\text{pass}} = k_p \tau(E_t) \sqrt{\Gamma(G_t)}, \quad (25)$$

$$S_{tij} = S_{tij}^{\text{act}} + S_t^{\text{pass}}. \quad (26)$$

The inverse-square term in (24) captures range attenuation of the emitted pattern. The passive term captures ambient or texture-assisted stereo cues that scale with exposure and gain.

The base quality score combines signal-to-noise ratio, ambient washout, motion, edge difficulty, and range saturation.

$$I_{tij}^{\text{amb}} = a_0 + a_1 \Omega_{tij}, \quad (27)$$

$$B_t^{\text{mot}} = \text{clip}(k_m \|\mathbf{v}_t\|_2 \tau(E_t), 0, b_{\max}), \quad (28)$$

$$W_{tij}^{\text{out}} = \frac{I_{tij}^{\text{amb}} \tau(E_t)}{S_{tij}^{\text{act}} + c_w}, \quad (29)$$

$$N_{tij}^{\text{shot}} = \frac{n_0(n_1 + n_2 \Gamma(G_t))}{S_{tij} + c_n}, \quad (30)$$

$$\chi_{tij} = \frac{S_{tij}}{b_0 + b_1 I_{tij}^{\text{amb}} + b_2 N_{tij}^{\text{shot}} + b_3 B_t^{\text{mot}}(b_4 + \delta_{tij})}, \quad (31)$$

$$R_{tij} = [\bar{Z}_{tij} / z_{\max} - r_0]_+, \quad (32)$$

$$Q_{tij}^0 = \sigma(\beta_s \chi_{tij} - \beta_w W_{tij}^{\text{out}} - \beta_e \delta_{tij} - \beta_r R_{tij}). \quad (33)$$

The variables I^{amb} , W^{out} , N^{shot} , and χ denote ambient infrared level, washout, noise proxy, and signal-to-noise proxy, respectively. All coefficients in (24)–(33) are contained in ϕ and are chosen to reproduce D455-like trends rather than device firmware exactly.

Localized physical regimes are then applied as differentiable penalties and bonuses.

$$C_{\rho,t} = B_{\rho_t}(p_t, e_t, g_t) - A_{\rho_t}(p_t, e_t, g_t), \quad (34)$$

$$Q_{tij} = \text{clip}(Q_{tij}^0 + \Omega_{tij} C_{\rho,t}, 0, 1). \quad (35)$$

For glare, A_ρ increases with exposure saturation, gain saturation, and insufficient projector power in the aperture halo, while B_ρ rewards the high-power, low-exposure, low-gain rescue region. For specular side walls, A_ρ increases with active-return blooming caused by high power, exposure, or gain, while B_ρ rewards conservative register settings that avoid multipath-like loss. For dark material, A_ρ is high unless exposure, gain, and projector power jointly recover the weak return; B_ρ adds a bounded rescue bonus, but the noise and motion penalties in (33) remain active.

The observed depth and observed quality are

$$V_{tij} = \mathbb{I}[Q_{tij} > q_0], \quad (36)$$

$$D_{tij}^{\text{obs}} = \bar{Z}_{tij} V_{tij}, \quad Q_{tij}^{\text{obs}} = Q_{tij} V_{tij}. \quad (37)$$

The policy consumes D_{tij}^{obs} ; Q_{tij}^{obs} remains a sensor-quality response used by downstream depth-health computation, teacher optimization, and diagnostic analysis rather than as an additional policy input. For gradient-based teacher optimization, the differentiable sensor layer uses a straight-through surrogate for the quality-to-depth gate and aggregates pixelwise camera-register gradients.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_t} = \sum_{i,j} \left(\frac{\partial \mathcal{L}}{\partial D_{tij}^{\text{obs}}} \frac{\partial D_{tij}^{\text{obs}}}{\partial \mathbf{c}_t} + \frac{\partial \mathcal{L}}{\partial Q_{tij}^{\text{obs}}} \frac{\partial Q_{tij}^{\text{obs}}}{\partial \mathbf{c}_t} \right). \quad (38)$$

The sensor layer evaluates (38) over all pixels and returns gradients for (P_t, E_t, G_t) . The raw depth $\bar{Z}_t$ and the projected mask Ω_t are treated as constants in this branch. This design keeps geometric ray intersection outside the camera-register gradient while making the active-depth image formation process differentiable with respect to the physical camera registers that the policy controls.

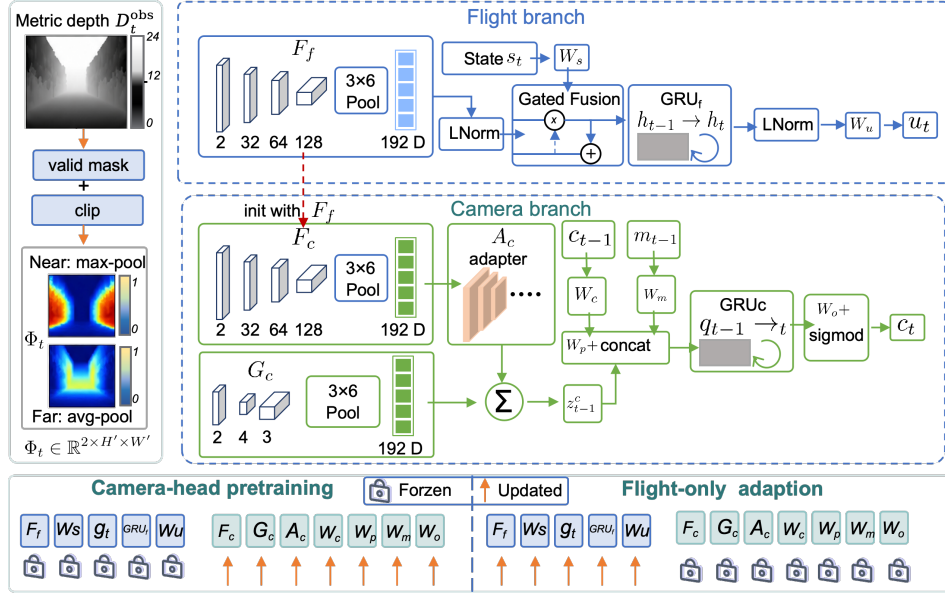


Fig. 4. The proposed split-stem recurrent policy architecture. The observed metric depth is converted into near-obstacle and far-opening channels. The flight branch fuses visual geometry with the flight state and outputs a local acceleration command. The camera branch uses an independent visual stem, a lightweight spatial adapter, the normalized camera state, and the local motion descriptor to output the next absolute camera target. Camera-head pretraining optimizes only camera-side parameters, while final flight-only adaptation freezes the camera stem and controller.

C. Split-Stem Policy

The policy is a split-stem recurrent architecture, as shown in Fig. 4. The design separates two roles that are coupled in the rollout but should not share all feature semantics during training. The flight branch must extract geometry useful for acceleration control, whereas the camera branch must preserve degradation cues needed to choose projector power, exposure, and gain. A single shared visual encoder would allow later flight-only adaptation to move the representation used by the camera controller, corrupting the semantics learned from the differentiable camera teacher. We therefore use one visual stem for flight, an independently parameterized copy for camera control, and a small camera-specific spatial adapter.

The observed depth is first mapped to a compact two-channel representation before entering either recurrent branch. Since invalid pixels are represented by zero-valued observations after the active-depth sensor model, define

$$V_{tij}^d = \mathbb{I}[D_{tij}^{\text{obs}} \geq z_{\min}], \quad (39)$$

$$\bar{D}_{tij} = \begin{cases} \text{clip}(D_{tij}^{\text{obs}}, z_{\min}, z_{\max}), & V_{tij}^d = 1, \\ z_{\max}, & V_{tij}^d = 0. \end{cases} \quad (40)$$

Two channels are then constructed.

$$\phi_t^{\text{near}} = \text{Pool}_{\max}^{H_d \times W_d} \left[V_t^d \odot \text{clip} \left(\frac{1/\bar{D}_t - 1/z_{\max}}{1/z_{\min} - 1/z_{\max}}, 0, 1 \right) \right], \quad (41)$$

$$\phi_t^{\text{far}} = \text{Pool}_{\text{avg}}^{H_d \times W_d} \left[V_t^d \odot \text{clip} \left(\frac{\bar{D}_t - z_{\min}}{z_{\max} - z_{\min}}, 0, 1 \right) \right], \quad (42)$$

$$\Phi_t = 2[\phi_t^{\text{near}}, \phi_t^{\text{far}}] - 1. \quad (43)$$

The near channel emphasizes collision-relevant foreground geometry by inverse depth and adaptive max pooling. The

far channel emphasizes open space and aperture cues by metric range and adaptive average pooling. In the experiments $H_d = 12$ and $W_d = 16$.

The flight visual stem F_f consists of three convolutional blocks with channels 32, 64, and 128, a stride-2 downsampling in the second block, adaptive average pooling to 3×6 , flattening, and a linear projection to 192 dimensions. Let LN denote layer normalization and φ denote LeakyReLU with slope 0.05. Then

$$\mathbf{z}_t^f = F_f(\Phi_t) \in \mathbb{R}^{192}, \quad (44)$$

$$\bar{\mathbf{z}}_t^f = \text{LN}(\mathbf{z}_t^f), \quad \bar{\mathbf{s}}_t = \text{LN}(W_s \mathbf{s}_t), \quad (45)$$

$$\mathbf{g}_t = \sigma \left(W_{g2} \varphi \left(W_{g1} [\bar{\mathbf{z}}_t^f, \bar{\mathbf{s}}_t] \right) \right), \quad (46)$$

$$\mathbf{r}_t = \varphi \left(\mathbf{g}_t \odot \bar{\mathbf{z}}_t^f + (1 - \mathbf{g}_t) \odot \bar{\mathbf{s}}_t \right). \quad (47)$$

The gate $\mathbf{g}_t$ is a learned elementwise mixture between visual geometry and the flight state. The recurrent flight state and action head use a gated recurrent unit (GRU) and are written as

$$\tilde{\mathbf{h}}_{t+1} = \text{GRU}_f(\mathbf{r}_t, \mathbf{h}_t), \quad (48)$$

$$\mathbf{h}_{t+1} = \text{LN} \left(\tilde{\mathbf{h}}_{t+1} + 0.1 R_h(\tilde{\mathbf{h}}_{t+1}) \right), \quad (49)$$

$$\mathbf{u}_t = W_u \varphi(\mathbf{h}_{t+1}), \quad (50)$$

where R_h is a small residual multilayer perceptron (MLP) initialized near zero.

The camera branch uses an independent visual stem F_c with the same convolutional layout as F_f . It is initialized from F_f for representation compatibility, but the two stems do not share parameters after initialization. A lightweight spatial adapter G_c extracts low-capacity local quality cues with 4-channel

convolutions and 2×3 pooling. The camera image feature is

$$\mathbf{z}_t^{c,0} = F_c(\Phi_t), \quad (51)$$

$$\mathbf{z}_t^{c,s} = W_{sp} G_c(\Phi_t), \quad (52)$$

$$\mathbf{z}_t^c = \text{LN}\left(\mathbf{z}_t^{c,0} + 0.25A_c(\mathbf{z}_t^{c,0}) + 0.35\mathbf{z}_t^{c,s}\right), \quad (53)$$

where A_c is the residual camera image adapter. The camera input excludes the goal vector. It uses only the image feature, the normalized current camera state $\hat{\mathbf{c}}_t = 2\mathbf{c}_t - 1$, and a motion descriptor

$$\mathbf{r}_t^{\text{up}} = \mathbf{R}_t \mathbf{e}_3, \quad \mathbf{m}_t = [\text{clip}(\mathbf{v}_t^{\text{local}}/v_{\max}, -2, 2), \mathbf{r}_t^{\text{up}}] \in \mathbb{R}^6, \quad (54)$$

where $\mathbf{e}_3 = [0, 0, 1]^\top$ and $\mathbf{r}_t^{\text{up}}$ is the body up direction expressed in the world frame. The camera recurrent controller is

$$\mathbf{e}_t^c = \text{LN}(W_c \hat{\mathbf{c}}_t), \quad \mathbf{e}_t^m = \text{LN}(W_m \mathbf{m}_t), \quad (55)$$

$$\mathbf{a}_t^c = \varphi(W_p[\mathbf{z}_t^c, \mathbf{e}_t^c, \mathbf{e}_t^m]), \quad (56)$$

$$\mathbf{q}_{t+1} = \text{LN}(\text{GRU}_c(\mathbf{a}_t^c, \mathbf{q}_t)), \quad (57)$$

$$\mathbf{c}_{t+1}^* = \sigma(W_o \varphi(\mathbf{q}_{t+1})). \quad (58)$$

The dimensions are 24 for the camera-state embedding, 24 for the motion-state embedding, and 96 for the camera GRU. The output $\mathbf{c}_{t+1}^* \in [0, 1]^3$ is an absolute target for projector power, exposure, and gain; the temporal smoothing in (8) is applied outside the network.

The same architectural partition is enforced during optimization. During camera-head pretraining, all non-camera parameters are held fixed and the flight recurrent state is treated as a detached context variable. During the final flight-only stage, all camera-side modules remain active in the forward pass but are excluded from parameter updates.

$$\Theta_{\text{cam}} = \{F_c, G_c, W_{sp}, A_c, W_c, W_m, W_p, \text{GRU}_c, W_o, \text{camera normalization layers}\}. \quad (59)$$

D. Teacher Optimization

The camera teacher is a state-wise differentiable perception optimizer. It does not prescribe the flight action. Instead, at a visited simulator state $\xi_t = (\mathbf{x}_t, \mathbf{c}_t, \rho_t)$, it asks which next camera register would make the local depth observation reliable while remaining physically economical and temporally smooth. The vehicle pose, velocity, scene, and raw geometry are fixed during this local solve; only the next camera target is optimized. Let $\bar{\mathbf{c}} = [\bar{P}, \bar{E}, \bar{G}]^\top = \sigma(\boldsymbol{\eta}) \in [0, 1]^3$ be a candidate target parameterized by unconstrained logits $\boldsymbol{\eta}$. Re-rendering the active-depth response at the same state gives

$$(D^{\text{obs}}(\bar{\mathbf{c}}), Q^{\text{obs}}(\bar{\mathbf{c}})) = \mathcal{S}_\phi(D_t^{\text{raw}}, \bar{\mathbf{c}}, \rho_t, \mathbf{x}_t). \quad (60)$$

Because $\mathcal{S}_\phi$ is differentiable with respect to $\bar{\mathbf{c}}$, losses computed from the observed depth and sensor-quality response can be backpropagated directly to projector power, exposure, and gain.

The primary reliability statistic is a lower-tail patch fill rate. Let $R_p = 6$ and $C_p = 8$ be the pooling grid, and let $\beta \in (0, 1]$ be the lower-tail fraction. A depth-health operator $\mathcal{H}$ converts

the active-depth response into patch fill values and averages the K least reliable patches.

$$\mathbf{p}(\bar{\mathbf{c}}) = \mathcal{H}(D^{\text{obs}}(\bar{\mathbf{c}}), Q^{\text{obs}}(\bar{\mathbf{c}}); R_p, C_p), \quad (61)$$

$$K = \lceil \beta R_p C_p \rceil, \quad (62)$$

$$r_{\text{fill}}(\bar{\mathbf{c}}) = \frac{1}{K} \sum_{i \in \mathcal{B}_K(\mathbf{p})} p_i, \quad (63)$$

where $\mathcal{B}_K(\mathbf{p})$ indexes the K smallest elements of $\mathbf{p}$. This conditional-value-at-risk style statistic penalizes local holes in the depth map rather than allowing high-confidence background pixels to hide a failed task-critical bottleneck region.

The teacher objective is

$$\bar{\mathbf{c}}_{t+1} = \arg \min_{\bar{\mathbf{c}} \in [0, 1]^3} \mathcal{L}_{\text{teach}}(\bar{\mathbf{c}}),$$

$$\begin{aligned} \mathcal{L}_{\text{teach}} = & \lambda_f [r_{\min} - r_{\text{fill}}(\bar{\mathbf{c}})]_+^2 + \lambda_s \|\bar{\mathbf{c}} - \mathbf{c}_t\|_2^2 \\ & + \lambda_P [\bar{P} - P_0]_+^2 + \lambda_b \|\mathbf{v}_t\|_2^2 \tau(\bar{E})^2 \\ & + \lambda_G \bar{G}^2 + \lambda_0 \omega(\bar{\mathbf{c}}) \|\bar{\mathbf{c}} - \mathbf{c}_{\text{nom}}\|_2^2. \end{aligned} \quad (64)$$

Here $r_{\min}$ is the required fill level, P_0 is the nominal power baseline, $\tau(\bar{E})$ maps normalized exposure to physical integration time, and $\mathbf{c}_{\text{nom}}$ is the nominal camera setting. The terms have distinct physical roles. The fill penalty raises depth support in the least reliable image regions. The smoothness term avoids frame-to-frame camera jumps. The power term discourages unnecessary projector intensity above the baseline. The blur term penalizes long exposure during fast motion, and the gain term discourages amplified noise. The final recovery term returns the camera toward the nominal setting only after the depth map is already healthy.

$$\omega(\bar{\mathbf{c}}) = \text{sg} \left[\text{clip} \left(\frac{r_{\text{fill}}(\bar{\mathbf{c}}) - r_{\min}}{m_{\text{fill}}}, 0, 1 \right) \right], \quad (65)$$

where m_{fill} is a recovery margin and $\text{sg}[\cdot]$ denotes a stop-gradient operation. Thus, nominal recovery does not weaken the gradient that first restores depth fill health.

The essential point is that the teacher label is produced by differentiating through the active-depth image formation model.

$$\nabla_{\boldsymbol{\eta}} \mathcal{L}_{\text{teach}} = \lambda_f \frac{\partial [r_{\min} - r_{\text{fill}}]_+^2}{\partial r_{\text{fill}}} \frac{\partial r_{\text{fill}}}{\partial \bar{\mathbf{c}}} \frac{\partial \bar{\mathbf{c}}}{\partial \boldsymbol{\eta}} + \nabla_{\boldsymbol{\eta}} \mathcal{L}_{\text{reg}}. \quad (66)$$

Here $\mathcal{L}_{\text{reg}}$ denotes the smoothness, power, blur, gain, and nominal terms in (64). The blur and gain terms are analytic functions of $\|\mathbf{v}_t\|_2 \tau(\bar{E})$ and $\bar{G}^2$, respectively; the fill term obtains its camera derivative through $\mathcal{S}_\phi$ and the downstream depth-health computation. Without this differentiable perception path, the camera label would have to be designed by hand.

The logits are initialized from the current camera state and optimized by the Adam optimizer.

$$\begin{aligned} \boldsymbol{\eta}^0 &= \text{logit}(\text{clip}(\mathbf{c}_t, \epsilon, 1 - \epsilon)), \\ \boldsymbol{\eta}^{k+1} &= \text{Adam}(\boldsymbol{\eta}^k, \nabla_{\boldsymbol{\eta}^k} \mathcal{L}_{\text{teach}}(\sigma(\boldsymbol{\eta}^k))), \\ \bar{\mathbf{c}}_{t+1} &= \sigma(\boldsymbol{\eta}^{K_{\text{teach}}}). \end{aligned} \quad (67)$$

For the reported DAGger relabeling, we use $K_{\text{teach}} = 120$ Adam steps with learning rate 0.08. During online relabeling, the next observation is generated by the learned camera state rather than by a teacher-smoothed camera. This makes the

Algorithm 1 State-Wise Teacher Collection and Camera Pre-training

Require: frozen flight policy Π_{θ_0} , scenarios $\mathcal{R}$, rollouts N , horizon T

Ensure: camera-pretrained policy θ_{pre}

- 1: $\mathcal{D}_0 \leftarrow \emptyset$
- 2: **for** $\rho \in \mathcal{R}$ **do**
- 3: **for** $n = 1, \dots, N$ **do**
- 4: reset the simulator under sensing regime ρ
- 5: initialize $\mathbf{c}_0 = \mathbf{c}_{\text{nom}}$
- 6: **for** $t = 0, \dots, T-1$ **do**
- 7: render D_t^{obs} by (3)–(4)
- 8: build $\mathbf{s}_t$ and $\mathbf{m}_t$ by (5)–(6)
- 9: infer $\mathbf{u}_t$ using the frozen flight policy Π_{θ_0}
- 10: solve $\bar{\mathbf{c}}_{t+1}$ by (64)–(67)
- 11: append $(D_t^{\text{obs}}, \mathbf{s}_t, \mathbf{c}_t, \mathbf{m}_t, \bar{\mathbf{c}}_{t+1}, \rho, x_t^{\text{loc}}, r_{\text{fill}})$ to $\mathcal{D}_0$
- 12: update the collection camera according to the collection policy and step dynamics with $\mathbf{u}_t$
- 13: **end for**
- 14: **end for**
- 15: **end for**
- 16: optimize only Θ_{cam} on $\mathcal{D}_0$ using (69)
- 17: **return** θ_{pre}

teacher labels match the distribution that the camera controller actually induces. Algorithm 1 summarizes the state-wise teacher collection and camera-pretraining procedure.

E. Camera Supervision and DAgger Relabeling

The teacher collection yields a sequence dataset

$$\mathcal{D} = \{(D_{b,t}^{\text{obs}}, \mathbf{s}_{b,t}, \mathbf{c}_{b,t}, \mathbf{m}_{b,t}, \bar{\mathbf{c}}_{b,t+1}, \rho_b, x_{b,t}^{\text{loc}}, r_{\text{fill},b,t})\}_{b,t}. \quad (68)$$

The camera branch is trained as a recurrent regressor from the policy observations and camera-side state to the differentiable teacher target. With the hidden states unrolled over each sequence, the supervised objective is

$$\begin{aligned} \mathcal{L}_{\text{sup}} = & \frac{1}{BT} \sum_{b=1}^B \sum_{t=0}^{T-1} \rho_{\delta}(\mathbf{c}_{b,t+1}^* - \bar{\mathbf{c}}_{b,t+1}) \\ & + \lambda_{\Delta c} \frac{1}{B(T-1)} \sum_{b=1}^B \sum_{t=1}^{T-1} \|\mathbf{c}_{b,t+1}^* - \mathbf{c}_{b,t}^*\|_2^2, \end{aligned} \quad (69)$$

where ρ_{δ} is the SmoothL1 penalty applied component-wise to projector power, exposure, and gain. During this stage, $\Theta \setminus \Theta_{\text{cam}}$ is fixed, and the flight recurrent state is treated as context rather than as a trainable path. The optimization therefore transfers the differentiable teacher’s sensor semantics into the camera controller without disturbing the flight controller.

Offline camera pretraining alone is not sufficient, because the camera head changes the future observations that it will later receive. Let $\mu(\theta_{\text{cam}})$ denote the rollout distribution induced by a learned camera controller. A camera trained only on $\mathcal{D}_0$ is queried under $\mu(\theta_{\text{cam}})$ during closed-loop execution,

Algorithm 2 DAgger Relabeling and Flight-Only Adaptation

Require: camera-pretrained policy θ_{pre} , teacher objective (64)

Ensure: final split-stem policy θ^*

DAgger relabeling

- 1: $\mathcal{D}_{\text{dag}} \leftarrow \emptyset$
- 2: **for** each learned-camera rollout **do**
- 3: **for** $t = 0, \dots, T-1$ **do**
- 4: render the current observation using the student camera state
- 5: infer $(\mathbf{u}_t, \mathbf{c}_{t+1}^*)$ with $\Pi_{\theta_{\text{pre}}}$
- 6: label the visited state $\bar{\mathbf{c}}_{t+1} \leftarrow \arg \min_{\bar{\mathbf{c}}} \mathcal{L}_{\text{teach}}$
- 7: append the labeled sample to $\mathcal{D}_{\text{dag}}$
- 8: update $\mathbf{c}_{t+1}$ by (8) and step dynamics with $\mathbf{u}_t$
- 9: **end for**
- 10: **end for**
- 11: retrain Θ_{cam} on $\mathcal{D}_{\text{dag}}$ using (69)

Flight-only adaptation

- 12: freeze Θ_{cam} and detach the camera recurrent path
- 13: optimize $\Theta_f = \Theta \setminus \Theta_{\text{cam}}$ by (72) under learned-camera rollouts
- 14: **return** θ^*

which generally differs from the teacher-smoothed or fixed-camera collection distribution. We correct this covariate shift with DAgger-style online relabeling at the sensor level.

$$\mathcal{D}_{\text{dag}} = \left\{ (\xi_t, \bar{\mathbf{c}}_{t+1}) \mid \xi_t \sim \mu(\theta_{\text{pre}}), \bar{\mathbf{c}}_{t+1} = \arg \min_{\bar{\mathbf{c}}} \mathcal{L}_{\text{teach}} \right\}. \quad (70)$$

In this rollout, the student camera output is applied through (8) and fully determines the next depth observation. The teacher is queried on the states actually visited by the student-induced camera distribution, so the relabels target the distribution that the learned camera will see during evaluation. The final relabeled dataset contains 144 sequences and 11,520 state-wise labels. Its teacher mean/std for $P/E/G$ are 0.584/0.408/0.403 and 0.164/0.197/0.181, respectively, indicating that the labels do not collapse to a nominal command and retain scene-dependent active-sensing variation. Algorithm 2 summarizes the online relabeling stage and the subsequent flight-only adaptation.

F. Flight Adaptation Objective

After DAgger, the camera branch defines a stable active-sensing policy. The final stage then adapts only the flight branch to the observation distribution created by that learned camera. This design is deliberate: gradients from the navigation loss are allowed to improve flight control, but they are not allowed to overwrite the camera semantics learned from the differentiable perception teacher. Formally, let $\Theta_f = \Theta \setminus \Theta_{\text{cam}}$. During flight-only adaptation,

$$\mathbf{c}_{t+1}^* = \text{sg}[\Pi_{\theta_{\text{cam}}}^c(D_t^{\text{obs}}, \mathbf{c}_t, \mathbf{m}_t, \mathbf{q}_t)], \quad \theta_{\text{cam}} \text{ fixed}, \quad (71)$$

where $\Pi_{\theta_{\text{cam}}}^c$ denotes the camera branch of Π_{θ} . The camera still affects the forward rollout through (4) and (8), but its parameters and recurrent state do not receive gradients.

The flight loss is

$$\begin{aligned} \mathcal{L}_{\text{flight}}(t) &= \lambda_v \mathcal{L}_v(t) + \lambda_a \mathcal{L}_{\text{acc}}(t) \\ &\quad + \lambda_j \mathcal{L}_{\text{jerk}}(t) + \lambda_c \mathcal{L}_{\text{collide}}(t), \\ \theta^* &= \arg \min_{\Theta_f} \mathbb{E}_{\rho, \mathbf{x}_0} \sum_{t=0}^{T-1} \mathcal{L}_{\text{flight}}(t). \end{aligned} \quad (72)$$

The camera parameter set Θ_{cam} is frozen throughout this optimization. The velocity term tracks a local goal-directed reference velocity over a short temporal window,

$$\mathcal{L}_v(t) = \rho_\delta \left(\left\| \frac{1}{W} \sum_{i=0}^{W-1} \mathbf{v}_{t-i} - \mathbf{v}_t^{\text{ref}} \right\|_2 \right), \quad (73)$$

while the control-effort and smoothness terms are

$$\mathcal{L}_{\text{acc}}(t) = \|\mathbf{u}_t\|_2^2, \quad \mathcal{L}_{\text{jerk}}(t) = \left\| \frac{\mathbf{u}_t - \mathbf{u}_{t-1}}{\Delta t_t} \right\|_2^2. \quad (74)$$

Let $d_{t,k}$ be the signed clearance, after subtracting the vehicle safety margin, for the k -th short-horizon obstacle query. Negative values indicate penetration. The collision barrier is

$$\mathcal{L}_{\text{collide}}(t) = \frac{1}{K_o} \sum_{k=1}^{K_o} \zeta_{t,k} \text{softplus}(-\gamma d_{t,k}), \quad (75)$$

where γ controls the sharpness and $\zeta_{t,k}$ weights samples that are moving toward the obstacle more heavily. No camera regularization is needed in this stage because camera-side parameters are not optimized. The resulting optimization separates the two roles: differentiable perception produces and stabilizes the active camera policy, and differentiable dynamics adapts the flight policy to exploit the depth stream created by that camera.

III. EXPERIMENTAL SETUP

A. Evaluation Rationale

The experiments are organized around the central differentiability claim in a controlled aperture-crossing benchmark. If the proposed sensor model closes the perception-simulation-control loop, then three outcomes should appear together. First, active camera control should improve closed-loop navigation compared with fixed or task-agnostic camera settings. Second, the improvement should be accompanied by healthier depth observations, because the mechanism is improved sensing rather than arbitrary flight regularization. Third, the camera response should be scene-specific in the task-critical bottleneck phase: glare should suppress exposure and gain, whereas low-reflectance material should increase them.

This rationale determines the organization of the evaluation. Overall success and collision quantify the task-level endpoint of the gradient chain. Depth fill measures whether the observation stream remains usable. Phase-wise camera statistics and matched-pose depth sequences test whether the learned behavior is consistent with the intended active-sensing mechanism rather than a trajectory artifact.

Setting	Value
Control frequency	15 Hz
Episode horizon	60 steps
Rendered depth resolution	96×72
Policy depth resolution	48×36
Camera action	power / exposure / gain in $[0, 1]^3$
Scenes	glare, dark, specular
Aperture center	randomized laterally
Scene yaw	randomized
Teacher rollouts per scene	4
Teacher batch size	12
Teacher optimization steps	120
Camera pretraining epochs	180, then 120 after relabeling
Evaluation episodes	900 per primary method

TABLE I

CORE SIMULATION, TRAINING, AND EVALUATION SETTINGS. EVALUATION USES 300 EPISODES PER SCENE FOR EACH PRIMARY METHOD.

B. Benchmark Configuration

Each episode lasts 60 control steps at 15 Hz, so the closed-loop horizon spans four seconds. The raw depth renderer produces a 96×72 image, and the policy receives a 48×36 encoded depth input. The task uses the same wall, aperture, start, and goal geometry across all sensing regimes. The aperture center is sampled laterally, and the entire scene can be randomly rotated in yaw. This rotation means that success requires using the local depth and state cues rather than a fixed world-frame direction. Representative instances from the glare, dark, and specular evaluation distribution are shown in Fig. 5; they illustrate the scene variation that makes context-dependent camera adaptation necessary.

The camera action is three-dimensional and normalized to $[0, 1]^3$. The fixed-camera baseline uses the nominal setting. The random-fixed baseline samples a static camera setting for each episode and keeps it fixed. Learned-camera methods update the camera state through the smoothed camera command described above. Table I summarizes the shared simulation, teacher, and evaluation settings used by the primary comparisons.

C. Compared Policies

The primary comparison isolates different ways of handling camera parameters, as summarized in Table II. The proposed method uses the full pipeline: differentiable camera relabeling, DAgger-style online relabeling, split-stem camera freezing, and flight-only adaptation. Fixed-camera tests whether a single nominal setting is sufficient. Random-fixed camera tests whether exposing the flight policy to diverse static camera settings alone explains the result. The non-differentiable learned-camera baseline tests whether a learned camera controller without the differentiable active-sensing training signal develops the same scene-specific behavior. The blind baseline removes depth information and provides a lower bound on geometry-free flight.

D. Metrics and Statistical Reporting

An episode is successful when the vehicle reaches the goal region without collision. Collision is recorded if the trajectory

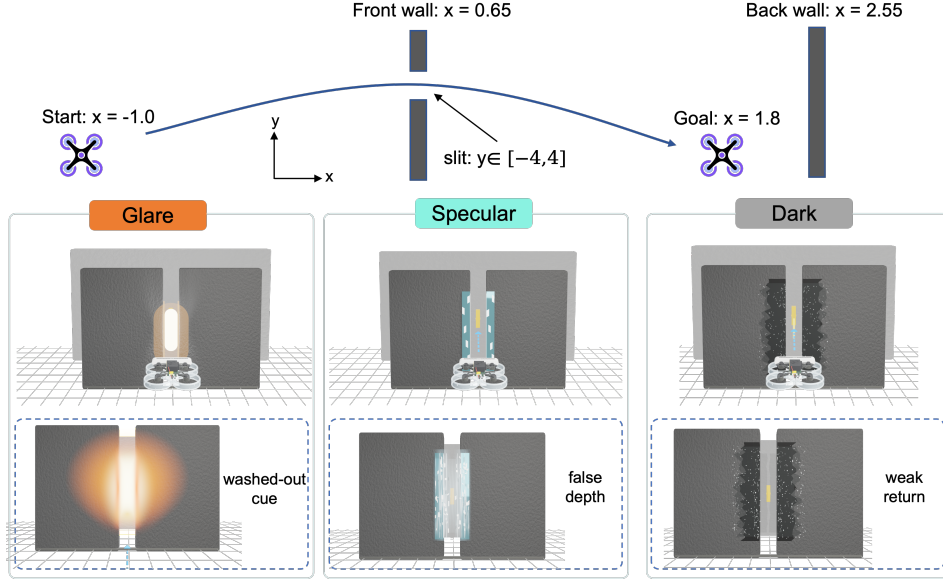


Fig. 5. Example scenes from the evaluation distribution. The benchmark samples glare, dark-material, and specular regimes, requiring the policy to adapt camera behavior to the flight context.

Method	Role	Description
Ours	Main model	split-stem learned camera with flight-only adaptation
Fixed-camera	Baseline	nominal constant camera setting
Random-fixed	Baseline	episode-constant random camera setting
Non-diff.	Baseline	learned camera without differentiable sensor training
Blind	Baseline	zero-depth controller
Pretrain	Diagnostic	camera-pretrained policy before flight adaptation
Dagger	Diagnostic	relabeled camera policy before flight adaptation

TABLE II

POLICIES USED IN THE QUANTITATIVE COMPARISON AND DIAGNOSTIC ANALYSIS. ONLY THE FIRST FIVE METHODS ARE PRIMARY NAVIGATION BASELINES; PRETRAIN AND DAGGER ARE INCLUDED TO SEPARATE CAMERA SEMANTICS FROM FLIGHT ADAPTATION.

intersects the wall or violates the clearance proxy. We also report terminal distance to the goal and the mean valid-depth fill rate. Let $\mathbf{x}_t$ denote the drone position, F_t the fraction of valid depth pixels at time t , and ρ_t the minimum clearance to any obstacle. We summarize

$$\text{success} = \mathbb{I}[\|\mathbf{x}_T - \mathbf{x}_{\text{goal}}\|_2 \leq \delta \wedge \neg \text{collision}], \quad (76)$$

$$\text{collision} = \mathbb{I}[\exists t : \rho_t \leq \epsilon], \quad (77)$$

$$\text{fill} = \frac{1}{T} \sum_{t=1}^T F_t, \quad (78)$$

$$d_{\text{final}} = \|\mathbf{x}_T - \mathbf{x}_{\text{goal}}\|_2. \quad (79)$$

Higher values are better for success and fill, whereas lower values are better for collision and terminal distance. Binary outcomes are reported with Wilson 95% confidence intervals. Continuous episode metrics, including fill and terminal distance, are reported with bootstrap 95% confidence intervals.

Camera behavior is summarized within three local phases relative to the wall, where x_t denotes the scalar local-forward component of $\mathbf{x}_t$ after transforming the position into the wall

frame.

$$x_t \in \begin{cases} \text{before,} & x_t < -0.25 \text{ m,} \\ \text{near,} & |x_t| \leq 0.25 \text{ m,} \\ \text{after,} & x_t > 0.25 \text{ m.} \end{cases} \quad (80)$$

The near phase is task-critical because the depth image then determines whether the vehicle can align with the aperture and avoid the wall. We report episode-aggregated bottleneck-phase power, exposure, and gain, as well as the glare-dark camera separation measured by the mean absolute parameter difference. These phase-wise statistics are diagnostic rather than training targets.

IV. RESULTS

The evaluation is organized around the causal chain required by the proposed method: optimization should converge, active perception should improve the closed-loop outcome, the improvement should be scene-dependent rather than a generic increase in valid pixels, and the relabeled camera semantics should become useful only after flight adaptation.

A. Training Curves Serve as Convergence Diagnostics

Fig. 6 reports training loss, success, and collision curves for the comparison methods. Their role is diagnostic: they

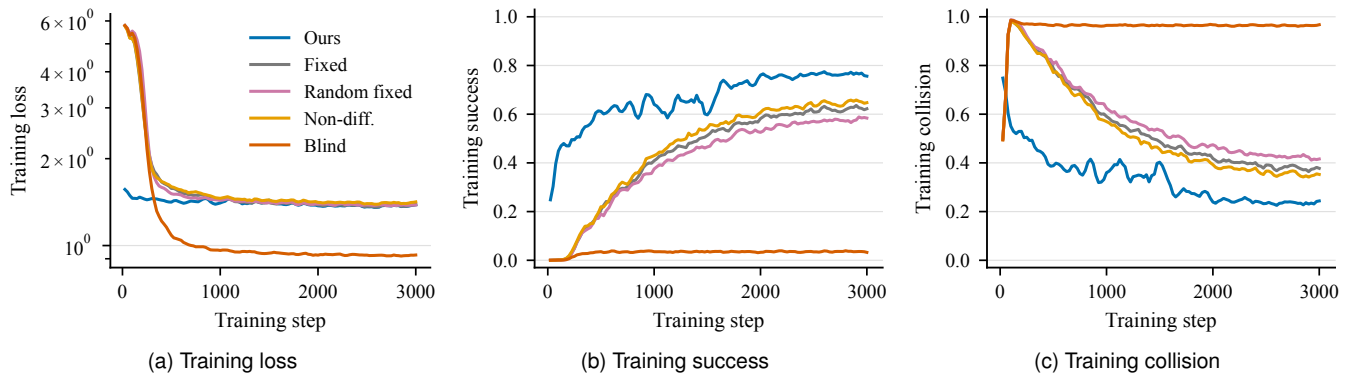


Fig. 6. Training diagnostics for the comparison policies. The curves document optimization behavior and convergence, whereas final claims use the held-out closed-loop evaluation summarized in Figs. 7–14.

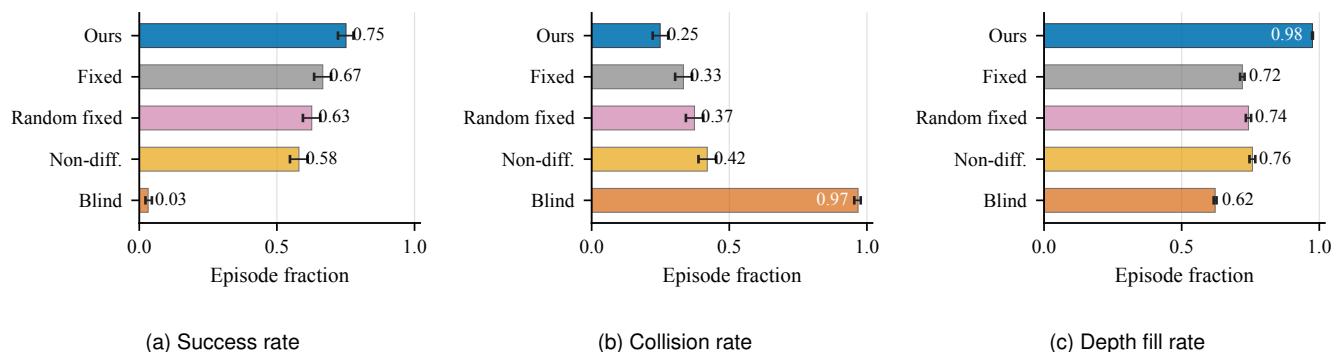


Fig. 7. Primary held-out navigation performance. Error bars denote 95% confidence intervals. The proposed policy improves success and depth fill while reducing collisions relative to the fixed-camera and non-differentiable learned-camera baselines.

show that the policies reached stable regimes and that the final flight-only run is not an incomplete optimization artifact. All claims below use held-out closed-loop evaluation with the same episode budget and scene distribution for every method.

For the final split-stem run, the training summary and held-out evaluation are consistent. The training summary reports approximately 0.750 success and 0.250 collision, while held-out evaluation gives 0.751 success and 0.249 collision. This agreement is important because the camera branch is frozen during flight-only adaptation; the flight policy adapts to a stable sensing distribution instead of chasing a moving camera representation.

B. Active Camera Control Improves Closed-Loop Navigation

Fig. 7 gives the primary held-out result. The proposed policy succeeds in 0.751 of episodes, compared with 0.667 for fixed-camera, 0.627 for random-fixed camera, 0.580 for the non-differentiable learned-camera baseline, and 0.032 for the blind controller. Collision decreases from 0.333 under fixed-camera to 0.249, and the valid-depth fill rate increases from 0.721 to 0.975. Error bars denote 95% confidence intervals: Wilson intervals for episode-level binary outcomes and bootstrap intervals for the continuous fill-rate metric.

The key point is not simply that the active camera produces more valid pixels. Fig. 8a plots the gain in depth fill against the gain in success, both measured relative to the fixed-camera

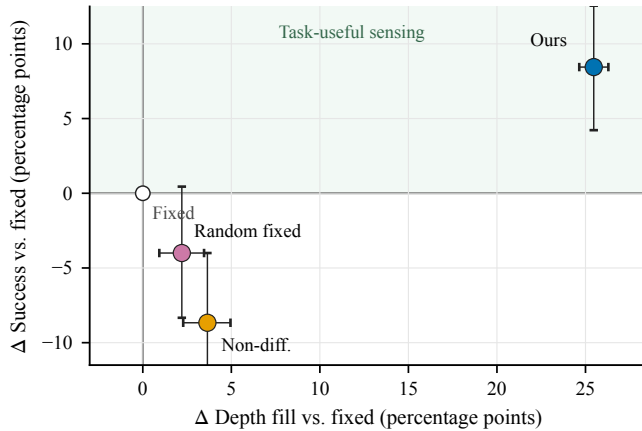
baseline in percentage points. The proposed method gains +25.4 percentage points in depth fill and +8.4 percentage points in success. Random-fixed and non-differentiable camera policies also recover some fill, but their success rates decrease. Thus, perception improvement becomes useful only when the recovered depth is conditioned on the scene and aligned with the downstream flight decision. The terminal-distance distribution in Fig. 9 is consistent with this interpretation: the proposed policy places more held-out episodes near the goal rather than merely avoiding collision.

C. Matched-Pose Depth Sequences Isolate the Camera Effect

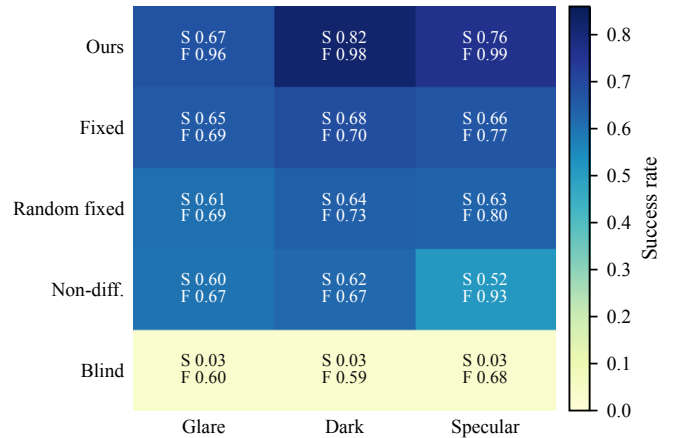
The qualitative depth sequences in Figs. 10, 11, and 12 provide a visual counterpart to the scalar metrics. Each panel re-renders observed depth at matched poses, using the same raw geometric depth but different camera settings. This matched-pose construction matters because it separates camera effects from trajectory effects: when the pose and scene geometry are fixed, changes in observed depth are attributable to active camera parameters under the same geometry.

D. Scene-Level Results Show Where the Gain Occurs

Fig. 8b shows that the benefit is not uniform across physical regimes. Dark material benefits the most: success and fill rise from 0.68 and 0.70 under fixed-camera to 0.82 and 0.98



(a) Fill–success coupling



(b) Scene-wise success/fill

Fig. 8. Navigation gains beyond scalar fill. (a) Changes are reported in percentage points relative to the fixed-camera baseline. Only the proposed policy lies in the regime where recovered depth is converted into higher closed-loop success. (b) Each cell reports success (S) and valid-depth fill (F); color encodes success rate.

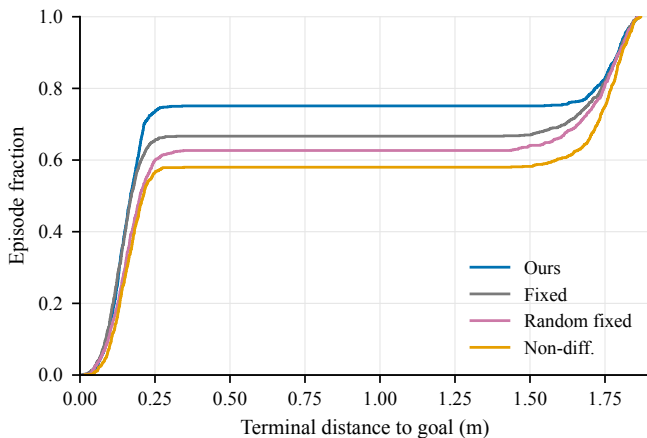


Fig. 9. Terminal distance distribution over held-out episodes. The proposed policy places more episode mass near the goal than the fixed, random-fixed, and non-differentiable learned-camera baselines.

under the proposed policy. This is a return-starvation regime, so increasing exposure and gain can recover weak returns without erasing the aperture boundary. Specular scenes are intermediate, improving from 0.66 and 0.77 to 0.76 and 0.99, because the camera can reduce local reflection artifacts but cannot fully remove edge instability. Glare remains hardest: fill improves from 0.69 to 0.96, yet success only moves from 0.65 to 0.67, because the residual error is concentrated near the aperture boundary.

This asymmetry is the mechanism-level result. Fill is necessary, but it is not sufficient. Dark and glare both gain fill; dark converts most of that gain into success because the recovered pixels preserve the aperture geometry, whereas glare can still corrupt the contrast exactly where the controller needs clean boundary evidence.

E. Scene-Specific Bottleneck Camera Behavior

Fig. 13 tests whether the learned camera has interpretable active-sensing semantics. In the bottleneck phase, the proposed

policy uses high power but low exposure and gain in glare ($P/E/G = 0.731/0.088/0.120$), high exposure and gain in dark material (0.665/0.694/0.683), and a lower-power intermediate setting in specular scenes (0.408/0.366/0.381). The direction of change matches the active-depth physics: glare requires suppressing integration and amplification to protect aperture contrast, dark material requires stronger integration and gain to recover weak return, and specular scenes benefit from lower power to reduce reflective hotspots.

The glare–dark camera separation reaches 0.412, whereas fixed-camera, non-differentiable learned-camera, and blind control remain at or near zero. Random-fixed camera has small separation because sampled settings vary across episodes, but that variation is not scene-conditioned. The comparison is therefore sharper than a fill-rate comparison: the proposed policy learns a physically meaningful sensing response, not merely a global bias toward more aggressive camera settings.

F. Camera Relabeling and Flight Adaptation Are Complementary

Fig. 14 separates two questions that are easy to conflate. The first is whether the camera branch learns meaningful scene-dependent semantics. The second is whether the flight branch can exploit the resulting observation distribution. Camera pretraining and DAGger relabeling answer the first question: they recover glare–dark camera separation in online rollouts. However, their closed-loop success remains low because the flight controller has not yet adapted to the learned-camera observation stream.

The final flight-only stage performs the second step while freezing the camera branch. It preserves the DAGger-learned sensing semantics and raises success to 0.751. The diagnostic checkpoints are therefore not failures of the camera teacher; they show that learning how to sense and learning how to fly under the new sensing distribution are coupled but distinct stages.

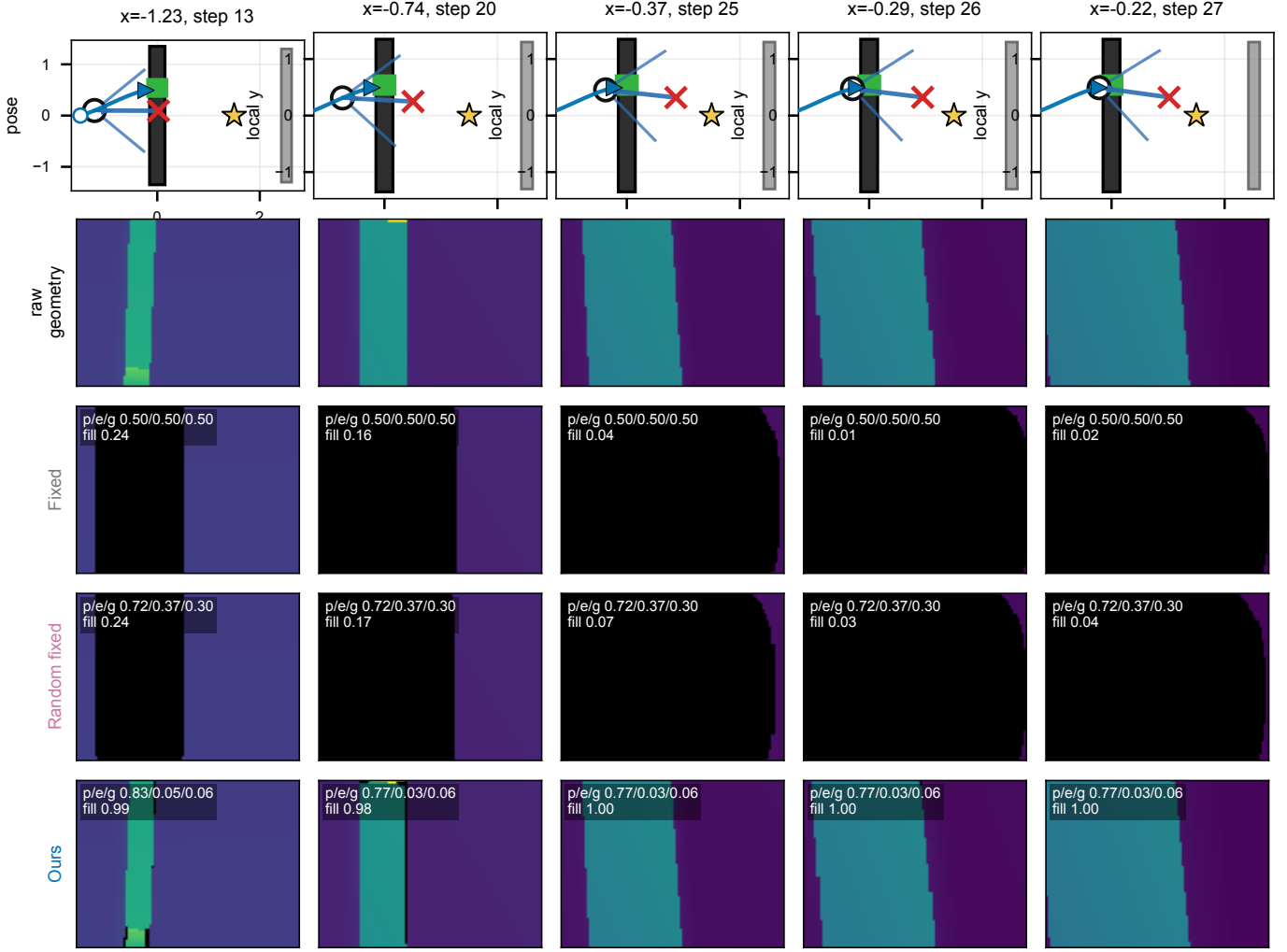


Fig. 10. Matched-pose depth observation sequences rendered along the same rollout poses in the glare regime. The raw geometry is fixed; only the camera parameters change the observed depth.

V. CONCLUSION

We presented a task-driven differentiable active depth sensing framework for drone navigation under degraded depth acquisition. The method treats projector power, exposure, and gain as closed-loop actions, learns camera behavior from differentiable teacher relabeling, and then adapts the flight branch while freezing the camera branch. In a controlled aperture-crossing validation task, the final policy improves success and depth fill over fixed, random-fixed, non-differentiable learned-camera, and blind baselines.

The evidence supports a focused mechanism claim. The proposed policy improves what the controller observes, changes camera settings in scene-dependent directions near the aperture, and converts part of the recovered depth support into higher closed-loop success. The result should not be read as a claim that depth fill alone determines navigation quality: glare still improves more in fill than in success, and the random-fixed and non-differentiable baselines can recover pixels without producing the same scene-conditioned navigation gain.

The current validation is simulation-only and uses one aperture-crossing family with one training seed. The sensor

model preserves causal relationships among power, exposure, gain, blur, noise, and localized degradation, but it is not a pixel-accurate model of a specific commercial depth camera. Extending the result to calibrated hardware, richer scene families, and multi-seed training remains the next validation step.

REFERENCES

- [1] R. Newbury, J. Collins, K. He, J. Pan, I. Posner, D. Howard, and A. Cosgun, “A Review of Differentiable Simulators,” Jul. 2024, arXiv preprint arXiv:2407.05560.
- [2] X. Zhang, R. Wang, Y. Ren, J. Sun, H. Fang, J. Chen, and G. Wang, “DiffAero: A GPU-Accelerated Differentiable Simulation Framework for Efficient Quadrotor Policy Learning,” Sep. 2025, arXiv preprint arXiv:2509.10247.
- [3] Y. Zhang, Y. Hu, Y. Song, D. Zou, and W. Lin, “Learning vision-based agile flight via differentiable physics,” *Nature Machine Intelligence*, vol. 7, no. 6, pp. 954–966, Jun. 2025.
- [4] J. Heeg, Y. Song, and D. Scaramuzza, “Learning Quadrotor Control from Visual Features Using Differentiable Simulation,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, May 2025, pp. 4033–4039.
- [5] A. Romero, E. Aljalbout, Y. Song, and D. Scaramuzza, “Actor-Critic Model Predictive Control: Differentiable Optimization meets Reinforcement Learning,” Feb. 2025, arXiv preprint arXiv:2306.09852.

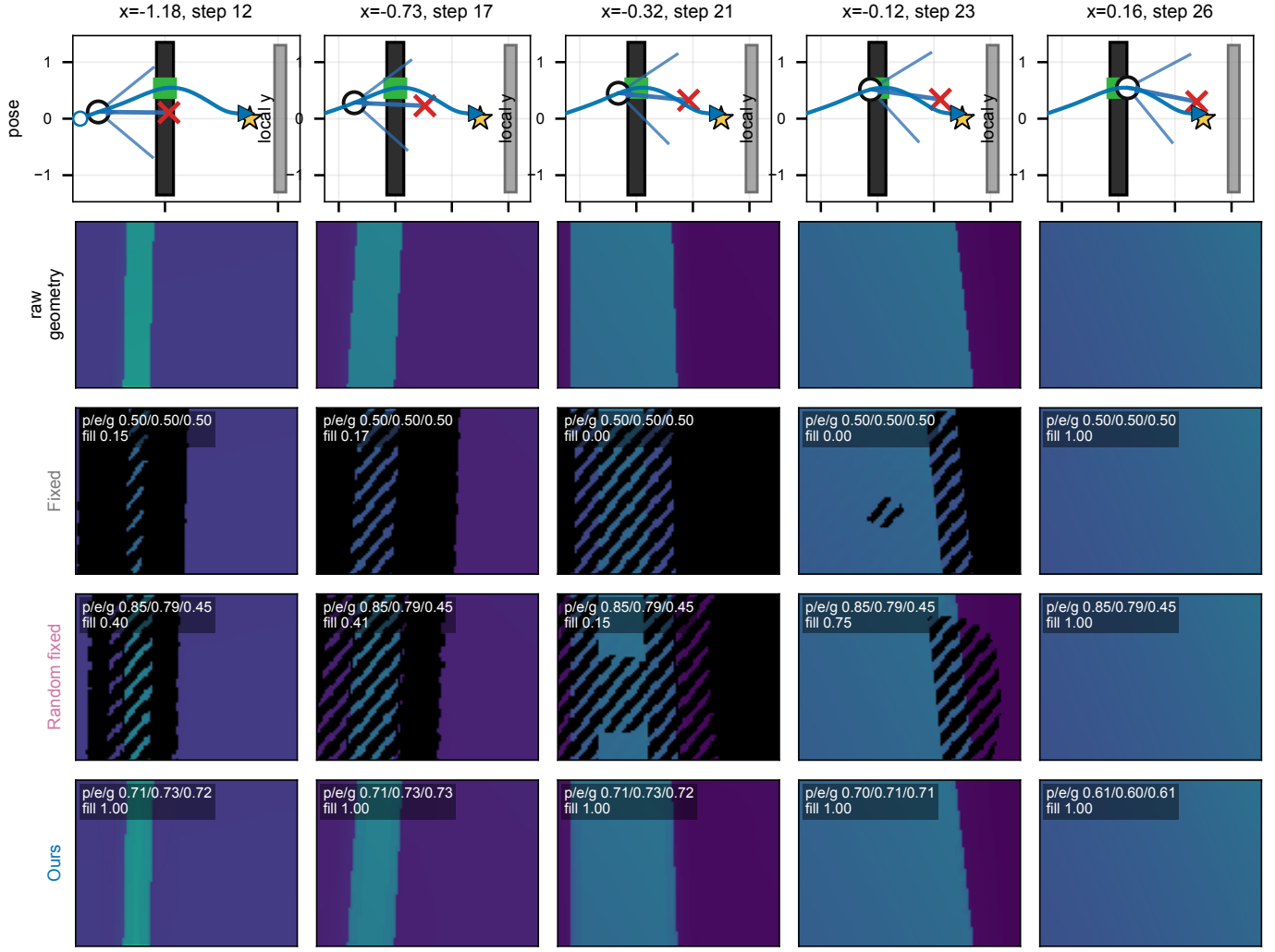


Fig. 11. Matched-pose depth observation sequences rendered along the same rollout poses in the dark-material regime. The raw geometry is fixed; only the camera parameters change the observed depth.

- [6] M. Denninger, D. Winkelbauer, M. Sundermeyer, W. Boerdijk, M. Knauer, K. H. Strobl, M. Humt, and R. Triebel, “BlenderProc2: A procedural pipeline for photorealistic rendering,” *Journal of Open Source Software*, vol. 8, no. 82, p. 4901, 2023.
- [7] A. Amini, T.-H. Wang, I. Gilitschenski, W. Schwarting, Z. Liu, S. Han, S. Karaman, and D. Rus, “VISTA 2.0: An open, data-driven simulator for multimodal sensing and policy learning for autonomous vehicles,” in *2022 IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 2419–2426.
- [8] X. Puig, E. Undersander, A. Szot, M. Dallahire Cote, T.-Y. Yang, R. Partsey, R. Desai, A. W. Clegg, M. Hlavac, S. Y. Min, V. Vondrus, T. Gervet, V.-P. Berges, J. M. Turner, O. Maksymets, Z. Kira, M. Kalakrishnan, J. Malik, D. S. Chaplot, U. Jain, D. Batra, A. Rai, and R. Mottaghi, “Habitat 3.0: A co-habitat for humans, avatars and robots,” 2023, arXiv preprint arXiv:2310.13724.
- [9] A. Elmquist, R. Serban, and D. Negrut, “A sensor simulation framework for training and testing robots and autonomous vehicles,” *Journal of Autonomous Vehicles and Systems*, vol. 1, no. 2, p. 021001, 2021.
- [10] H. Blasinski, J. Farrell, T. Lian, Z. Liu, and B. Wandell, “Optimizing image acquisition systems for autonomous driving,” *Electronic Imaging*, vol. 2018, no. 5, pp. 161–1–161–7, 2018.
- [11] Z. Liu, T. Lian, J. Farrell, and B. A. Wandell, “Neural network generalization: The impact of camera parameters,” *IEEE Access*, vol. 8, pp. 10 443–10 454, 2020.
- [12] Z. Lyu, T. Goossens, B. A. Wandell, and J. Farrell, “Validation of physics-based image systems simulation with 3D scenes,” *IEEE Sensors Journal*, vol. 22, no. 20, pp. 19 400–19 410, 2022.
- [13] A. Carlson, K. A. Skinner, R. Vasudevan, and M. Johnson-Roberson, “Modeling camera effects to improve visual learning from synthetic data,” in *European Conference on Computer Vision Workshops*, 2018.
- [14] —, “Sensor transfer: Learning optimal sensor effect image augmentation for sim-to-real domain adaptation,” *IEEE Robotics and Automation Letters*, vol. 4, no. 3, pp. 2431–2438, 2019.
- [15] H. Ouyang, Z. Shi, C. Lei, K. L. Law, and Q. Chen, “Neural camera simulators,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 7696–7705.
- [16] B.-H. Chen, N. M. Batagoda, and D. Negrut, “DiffPhysCam: Differentiable Physics-Based Camera Simulation for Inverse Rendering and Embodied AI,” Aug. 2025, arXiv preprint arXiv:2508.08831.
- [17] C. Yan and D. G. Dansereau, “TaCOS: Task-Specific Camera Optimization with Simulation,” 2024, arXiv preprint arXiv:2404.11031.
- [18] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “NeRF: Representing scenes as neural radiance fields for view synthesis,” in *European Conference on Computer Vision*, 2020, pp. 405–421.
- [19] T. Müller, A. Evans, C. Schied, and A. Keller, “Instant neural graphics primitives with a multiresolution hash encoding,” *ACM Transactions on Graphics*, vol. 41, no. 4, pp. 1–15, 2022.
- [20] M. Tancik, E. Weber, E. Ng, R. Li, B. Yi, T. Wang, A. Kristoffersen, J. Austin, K. Salahi, A. Ahuja, D. McAllister, J. Kerr, and A. Kanazawa, “Nerfstudio: A modular framework for neural radiance field development,” in *ACM SIGGRAPH Conference Proceedings*, 2023.
- [21] B. Kerbl, G. Kopanas, T. Leimkuehler, and G. Drettakis, “3D gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1–14, 2023.

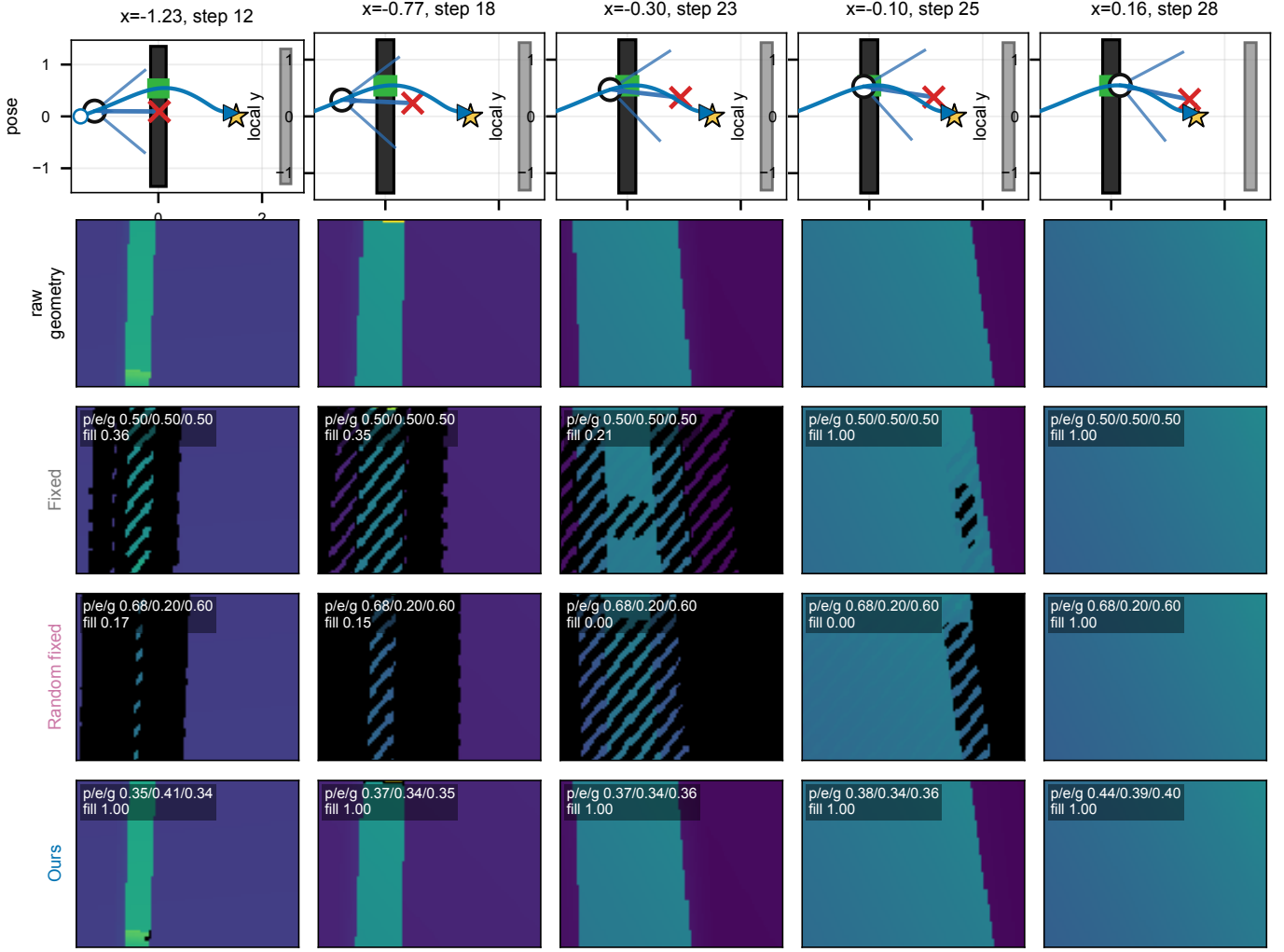


Fig. 12. Matched-pose depth observation sequences rendered along the same rollout poses in the specular regime. The raw geometry is fixed; only the camera parameters change the observed depth.

- [22] A. Guédon and V. Lepetit, “SuGaR: Surface-aligned gaussian splatting for efficient 3D mesh reconstruction and high-quality mesh rendering,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 5354–5363.
- [23] J. Hasselgren, N. Hofmann, and J. Munkberg, “Shape, light, and material decomposition from images using Monte Carlo rendering and denoising,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 22 856–22 869.
- [24] S. Pidhorskyi, T. Bagautdinov, S. Ma, J. Saragih, G. Schwartz, Y. Sheikh, and T. Simon, “Depth of field aware differentiable rendering,” *ACM Transactions on Graphics*, vol. 41, no. 6, 2022.
- [25] C. Sun, Y. Li, J. Li, C. Wang, and X. Dai, “CaSE-NeRF: Camera settings editing of neural radiance fields,” in *Computer Graphics International*, 2023, pp. 95–107.
- [26] Z. Wu, X. Li, J. Peng, H. Lu, Z. Cao, and W. Zhong, “DoF-NeRF: Depth-of-field meets neural radiance fields,” in *ACM International Conference on Multimedia*, 2022, pp. 1718–1729.
- [27] Y. Wang, S. Yang, Y. Hu, and J. Zhang, “NeRFocus: Neural radiance field for 3D synthetic defocus,” 2022, arXiv preprint arXiv:2203.05189.
- [28] M.-J. Kim, G. Gu, and J. Choo, “LensNeRF: Rethinking volume rendering based on thin-lens camera model,” in *IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 3182–3191.
- [29] R. Bajcsy, “Active perception,” *Proceedings of the IEEE*, vol. 76, no. 8, pp. 996–1005, 1988.
- [30] J. Aloimonos, I. Weiss, and A. Bandyopadhyay, “Active vision,” *International Journal of Computer Vision*, vol. 1, no. 4, pp. 333–356, 1988.
- [31] R. Bajcsy, J. Aloimonos, and J. K. Tsotsos, “Revisiting active perception,” *Autonomous Robots*, vol. 42, no. 2, pp. 177–196, 2018.
- [32] P. Florence, J. Carter, and R. Tedrake, “Integrated Perception and Control at High Speed: Evaluating Collision Avoidance Maneuvers Without Maps,” in *Algorithmic Foundations of Robotics XII*, K. Goldberg, P. Abbeel, K. Bekris, and L. Miller, Eds. Cham: Springer International Publishing, 2020, vol. 13, pp. 304–319.
- [33] Y. Song, K. Shi, R. Penicka, and D. Scaramuzza, “Learning Perception-Aware Agile Flight in Cluttered Environments,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, May 2023, pp. 1989–1995.
- [34] J. Tordesillas and J. P. How, “Deep-PANTHER: Learning-Based Perception-Aware Trajectory Planner in Dynamic Environments,” *IEEE Robotics and Automation Letters*, vol. 8, no. 3, pp. 1399–1406, Mar. 2023.
- [35] G. Zhao, T. Wu, Y. Chen, and F. Gao, “Learning Speed Adaptation for Flight in Clutter,” *IEEE Robotics and Automation Letters*, vol. 9, no. 8, pp. 7222–7229, Aug. 2024.
- [36] Y. Hu, Y. Zhang, Y. Song, Y. Deng, F. Yu, L. Zhang, W. Lin, D. Zou, and W. Yu, “Seeing Through Pixel Motion: Learning Obstacle Avoidance From Optical Flow With One Camera,” *IEEE Robotics and Automation Letters*, vol. 10, no. 6, pp. 5871–5878, Jun. 2025.
- [37] A. Loquercio, A. I. Maqueda, C. R. del Blanco, and D. Scaramuzza, “Deep drone racing: From simulation to reality with domain randomization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 1, pp. 133–140, 2018.
- [38] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, “Learning high-speed flight in the wild,” *Science Robotics*, vol. 6, no. 59, p. eab5217, 2021.
- [39] P. Foehn, E. Kaufmann, A. Romero, R. Penicka, S. Sun, L. Bauersfeld,

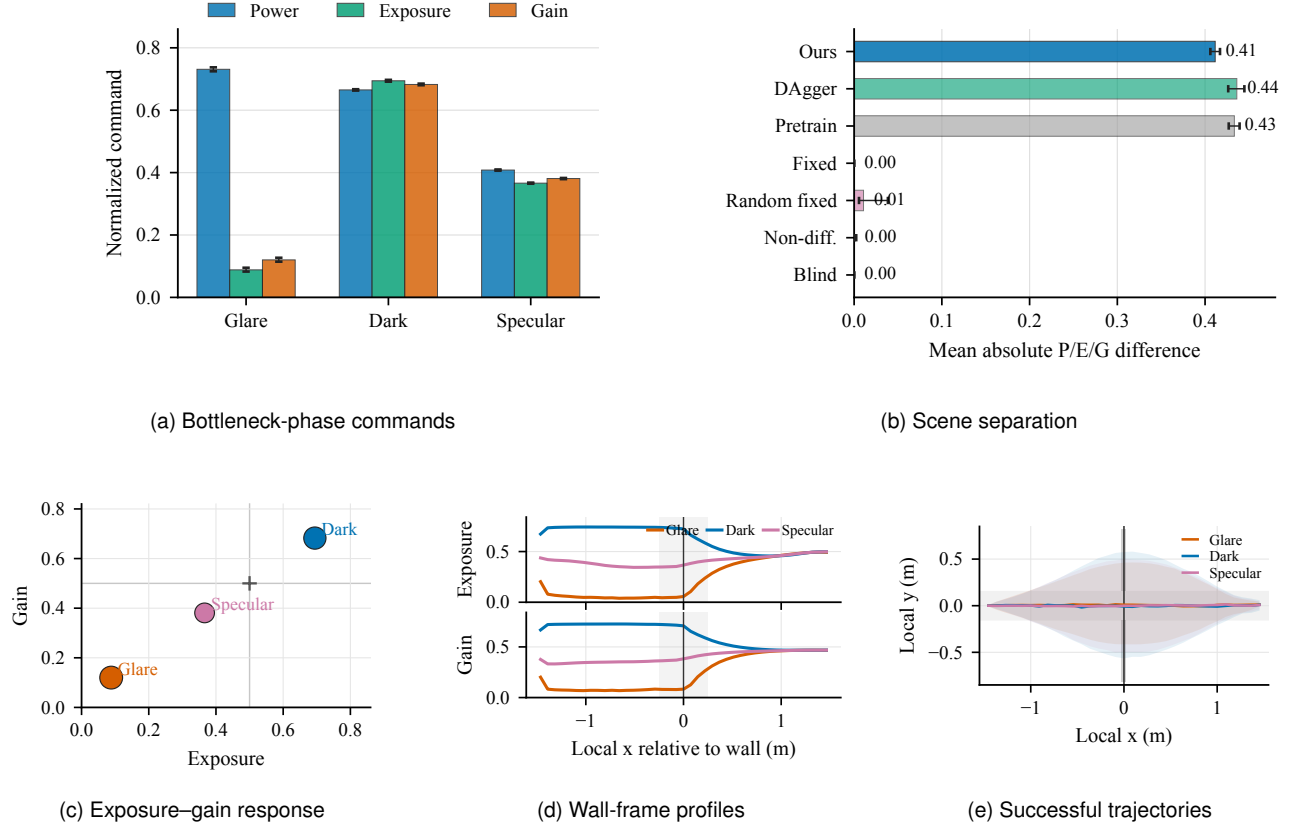


Fig. 13. Scene-conditioned active-camera mechanism during the bottleneck phase. The final policy separates glare, dark, and specular regimes in normalized power, exposure, and gain; the glare-dark separation is large only for the relabeled and final active-camera policies. The wall-frame profiles and successful trajectory envelopes connect these sensor actions to the aperture-crossing phase of the validation task.

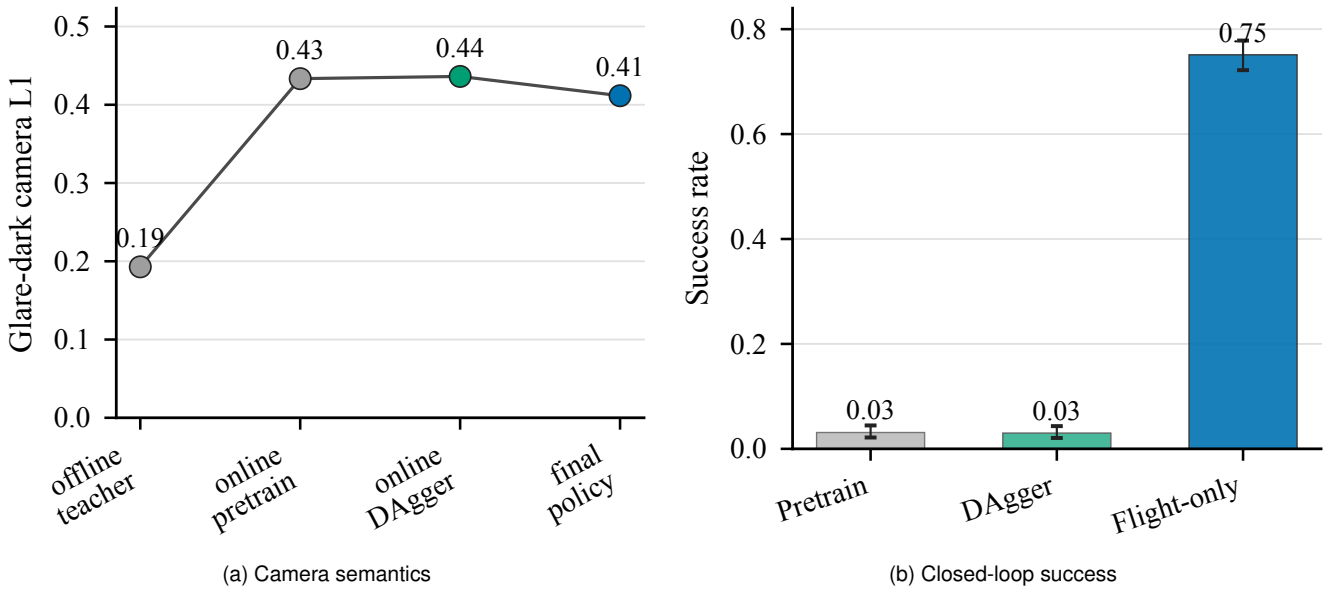


Fig. 14. DAGger relabeling and flight-only adaptation play different roles. Relabeling restores scene-specific camera semantics, while the final flight-only stage freezes the camera branch and trains the flight branch to use the new observation distribution.

- [41] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 2011, pp. 627–635.